

BANK LOAN OPERATION

Section 1 — JOINS & CASE

Q1. Customer Loan Application Report

Using appropriate JOINS, display:

- Customer Name
- Customer Segment
- Loan Product
- Branch Name
- Application Date
- Requested Loan Amount
- Application Status

```
SELECT c.customer_name, c.customer_segment,  
       lp.loan_product_name, b.branch_name,  
       la.application_date, la.requested_loan_amount, la.application_s  
tatus  
FROM fact_loan_application la  
INNER JOIN dim_customer c ON la.customer_id = c.customer_id  
INNER JOIN dim_branch b ON la.branch_id = b.branch_id  
INNER JOIN dim_loan_product lp ON la.loan_product_id = lp.loa  
n_product_id;
```

Data Output Messages Notifications							
Showing rows: 1 to 1000 Page No: 1 of 8							
	customer_name character varying (120)	customer_segment character varying (60)	loan_product_name character varying (100)	branch_name character varying (120)	application_date date	requested_loan_amount numeric (14,2)	application_status character varying (40)
1	Riya Nair 38	Mass Market	Personal Loan	Bengaluru Marathahalli Branch	2023-05-12	181991.00	Approved
2	Kavya Khan 75	Premium	Personal Loan	Ahmedabad Satellite Branch	2023-09-17	213982.00	Approved
3	Sneha Mehta 112	Mass Affluent	Personal Loan	Lucknow Gomti Nagar Branch	2024-01-23	245973.00	Approved
4	Aditi Verma 149	Mass Affluent	Personal Loan	Jaipur Vaishali Nagar Branch	2024-05-30	277964.00	Approved
5	Rahul More 186	Affluent	Personal Loan	Pune Kothrud Branch	2024-10-05	309955.00	Approved
6	Aditya Sharma 223	Affluent	Personal Loan	Chennai T Nagar Branch	2025-02-10	341946.00	Approved
7	Shreya Shah 260	Mass Market	Personal Loan	Delhi Dwarka Branch	2025-06-18	373937.00	Approved
8	Nisha Deshmukh 297	Mass Market	Personal Loan	Mumbai Thane Branch	2025-10-24	405928.00	Approved
9	Pooja Singh 334	Premium	Personal Loan	Bengaluru Indiranagar Branch	2023-03-02	437919.00	Approved
10	Priya Patil 371	Mass Affluent	Personal Loan	Ahmedabad Navrangpura Bra...	2023-07-08	469910.00	Approved
11	Siddharth Joshi 408	Mass Affluent	Personal Loan	Kolkata New Town Branch	2023-11-13	501901.00	Approved
12	Rohan Rao 445	Affluent	Personal Loan	Jaipur Malviya Nagar Branch	2024-03-20	533892.00	Approved
Total rows: 8000 Query complete 00:00:00.553 CRLF Ln 12, Col 1							

Q2. Approved Loan Portfolio

Display all approved loans with:

- Customer Name
- Branch
- Loan Product
- Requested Amount
- Sanctioned Amount
- Disbursement Date
- Loan Status

```
SELECT c.customer_name,
b.branch_name,
lp.loan_product_name,
la.requested_loan_amount,application_status,
lac.sanctioned_amount,lac.disbursement_date,lac.loan_status
FROM fact_loan_application la
INNER JOIN dim_customer c ON la.customer_id = c.customer_id
INNER JOIN dim_branch b ON la.branch_id = b.branch_id
INNER JOIN dim_loan_product lp ON la.loan_product_id = lp.loan_product_id
```

```

n_product_id
INNER JOIN fact_loan_account lac ON la.loan_application_id =
lac.loan_application_id
WHERE la.application_status = 'Approved';

```

Data Output Messages Notifications							
Showing rows: 1 to 1000 Page No: 1 of 6							
	customer_name character varying (120)	branch_name character varying (120)	loan_product_name character varying (100)	requested_loan_amount numeric (14,2)	application_status character varying (40)	sanctioned_amount numeric (14,2)	disbursement_date
1	Kavya Rao 3535	Pune Wakad Branch	Home Loan	4580913.00	Approved	4122821.70	2023-09-08
2	Aarav Patil 3461	Mumbai Andheri Branch	Home Loan	4434531.00	Approved	3991077.90	2026-01-04
3	Arjun Singh 3424	Delhi Connaught Place Branch	Home Loan	4361340.00	Approved	3663525.60	2025-08-27
4	Kunal Deshmukh 3387	Hyderabad Secunderabad Bra...	Home Loan	4288149.00	Approved	3859334.10	2025-04-19
5	Ananya Shah 3350	Pune Camp Branch	Home Loan	4214958.00	Approved	3793462.20	2024-12-10
6	Neha Sharma 3313	Jaipur C Scheme Branch	Home Loan	4141767.00	Approved	3727590.30	2024-08-20
7	Isha More 3276	Kolkata Park Street Branch	Home Loan	4068576.00	Approved	3417603.84	2024-04-12
8	Vivaan Mehta 3202	Bengaluru Yelahanka Branch	Home Loan	3922194.00	Approved	2941645.50	2023-07-27
9	Rohan Khan 3165	Mumbai Borivali Branch	Home Loan	3849003.00	Approved	3656552.85	2023-03-19
10	Siddharth Nair 3128	Delhi Rohini Branch	Home Loan	3775812.00	Approved	3171682.08	2025-11-08
11	Priya Gupta 3091	Hyderabad Kukatpally Branch	Home Loan	3702621.00	Approved	3110201.64	2025-07-01
12	Neha Nair 3053	Mumbai Navi Mumbai Branch	Home Loan	3629430.00	Approved	3048721.20	2025-02-18
13	Isha Gupta 3016	Delhi Noida Branch	Home Loan	3556239.00	Approved	3200615.10	2024-10-11
14	Meera Kulkarni 2979	Chennai OMR Branch	Home Loan	3483048.00	Approved	2612286.00	2024-06-03
Total rows: 5290 Query complete 00:00:00.651 CRLF Ln 41, Col 42							

Q3. Branch Approval Performance

For every branch calculate:

- Total Applications
- Approved Applications
- Rejected Applications
- Approval Rate %

Sort branches by **Approval Rate descending**.

```

-- - Total Applications
SELECT b.branch_name, COUNT(la.loan_application_id) AS total_a
pplication

```

```

from fact_loan_application la
left join dim_branch b ON la.branch_id = b.branch_id
group by b.branch_name
order by total_application DESC;

```

Data Output Messages Notifications

Showing rows: 1 to 45 Page No: 1 of 1

	branch_name character varying (120)	total_application bigint
1	Hyderabad Banjara Hills Branch	183
2	Delhi Saket Branch	183
3	Pune Pimpri Branch	182
4	Lucknow Gomti Nagar Branch	182
5	Bengaluru Marathahalli Branch	181
6	Bengaluru Whitefield Branch	181
7	Lucknow Hazratganj Branch	180
8	Lucknow Aliganj Branch	180
9	Delhi Dwarka Branch	180
10	Pune Hadapsar Branch	180
11	Mumbai Dadar Branch	180
12	Chennai OMR Branch	180
13	Delhi Noida Branch	179
14	Ahmedabad SG Highway Bran...	179
15	Delhi Laxmi Nagar Branch	178

Total rows: 45 Query complete 00:00:00.276 CRLF Ln 59, Col 33

```

-- - Approved Applications
SELECT b.branch_name, COUNT(la.application_status) AS approved
_application
from fact_loan_application la
left join dim_branch b ON la.branch_id = b.branch_id
where application_status='Approved'
group by b.branch_name
order by approved_application DESC;

```

	branch_name character varying (120)	approved_application bigint
1	Pune Hadapsar Branch	135
2	Bengaluru Koramangala Branch	130
3	Mumbai Andheri Branch	129
4	Chennai Anna Nagar Branch	129
5	Pune Pimpri Branch	129
6	Lucknow Aliganj Branch	128
7	Delhi Dwarka Branch	128
8	Delhi Rohini Branch	127
9	Pune Kothrud Branch	126
10	Delhi Saket Branch	124
11	Bengaluru Whitefield Branch	124

Total rows: 45 Query complete 00:00:00.186

```
-- - Rejected Applications
SELECT b.branch_name, COUNT(la.application_status) AS rejected
_application
from fact_loan_application la
left join dim_branch b ON la.branch_id = b.branch_id
where application_status='Rejected'
group by b.branch_name
order by rejected_application DESC;
```

	branch_name character varying (120)	rejected_application bigint
1	Delhi Noida Branch	65
2	Hyderabad Banjara Hills Branch	64
3	Ahmedabad Navrangpura Bra...	63
4	Kolkata Salt Lake Branch	63
5	Mumbai Navi Mumbai Branch	62
6	Kolkata Park Street Branch	62
7	Jaipur C Scheme Branch	61
8	Lucknow Gomti Nagar Branch	60
9	Delhi Laxmi Nagar Branch	60
10	Ahmedabad Maninagar Branch	60
11	Mumbai Powai Branch	59
Total rows: 45		Query complete 00:00:00.212

```
-- - Approval Rate %
-- Sort branches by Approval Rate descending.
SELECT b.branch_name, ROUND(
    (
        SELECT COUNT(*)
        FROM fact_loan_application la
        WHERE la.branch_id = b.branch_id
            AND la.application_status = 'Approved'
    ) * 100.0 / COUNT(la.loan_application_id),
    2
) AS approval_rate
FROM dim_branch b
```

```
LEFT JOIN fact_loan_application la ON b.branch_id = la.branch_id
GROUP BY b.branch_name, b.branch_id
ORDER BY approval_rate DESC;
```

Data Output	Messages	Notifications
≡ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈 SQL		
	branch_name character varying (120)	approval_rate numeric
1	Pune Hadapsar Branch	75.00
2	Bengaluru Koramangala Branch	73.03
3	Delhi Rohini Branch	72.57
4	Chennai Anna Nagar Branch	72.47
5	Mumbai Andheri Branch	72.47
6	Lucknow Aliganj Branch	71.11
7	Delhi Dwarka Branch	71.11
8	Pune Pimpri Branch	70.88
9	Mumbai Borivali Branch	70.86
10	Pune Kothrud Branch	70.79
11	Delhi Connaught Place Branch	69.54
Total rows: 45		Query complete 00:00:00.220

Q4. Customer Credit Rating

Using `CASE`, classify customers based on `credit_score`:

Credit Score	Rating
750+	Excellent
700–749	Good
650–699	Average

Credit Score	Rating
600–649	Poor
< 600	Very Poor

Display the original credit score and calculated rating.

```
SELECT customer_name, credit_score,
       (CASE
         WHEN credit_score < 600 THEN 'Very Poor'
         WHEN credit_score > 600 AND credit_score < 649 THEN 'Poor'
         WHEN credit_score > 650 AND credit_score < 699 THEN 'Average'
         WHEN credit_score > 700 AND credit_score < 749 THEN 'Good'
         WHEN credit_score > 750 THEN 'Excellent'
       END) AS Rating
FROM dim_customer;
```

115			
Data Output Messages Notifications			
≡ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈 SQL			
	customer_name character varying (120) 🔒	credit_score integer 🔒	rating text 🔒
1	Aarav Gupta 1	633	Poor
2	Vivaan Iyer 2	646	Poor
3	Aditya Joshi 3	701	Good
4	Arjun Singh 4	718	Good
5	Rohan Shah 5	725	Good
6	Rahul More 6	736	Good
7	Kunal Mehta 7	747	Good
8	Siddharth Nair 8	758	Excellent
9	Aditi Kulkarni 9	769	Excellent
10	Ananya Rao 10	780	Excellent
11	Priya Patil 11	791	Excellent
12	Sneha Deshmukh 12	802	Excellent
Total rows: 4000		Query complete 00:00:00.453	

Q5. Loan Size Classification

Using `CASE`, classify loans based on `sanctioned_amount`:

Amount	Category
< ₹5 Lakh	Small
₹5–15 Lakh	Medium
₹15–30 Lakh	Large
> ₹30 Lakh	Very Large

Display:

- Loan Account
- Customer
- Loan Product
- Sanctioned Amount
- Loan Category

```
SELECT lac.loan_account_id,c.customer_name,p.loan_product_name,lac.sanctioned_amount,
    (CASE
        WHEN lac.sanctioned_amount<500000 THEN 'Small'
        WHEN lac.sanctioned_amount>500000 AND lac.sanctioned_amount<1500000 THEN 'Medium'
        WHEN lac.sanctioned_amount>1500000 AND lac.sanctioned_amount<3000000 THEN 'Large'
        WHEN lac.sanctioned_amount>3000000 THEN 'Very Large'
    END) as loan_Category
from fact_loan_application la
inner join dim_customer c on c.customer_id =la.customer_id
inner join dim_loan_product p on p.loan_product_id =la.loan_product_id
inner join fact_loan_account lac on lac.loan_application_id = la.loan_application_id
order by sanctioned_amount asc;
```

Data Output Messages Notifications					
Showing rows: 1 to 1000					
	loan_account_id integer	customer_name character varying (120)	loan_product_name character varying (100)	sanctioned_amount numeric (14,2)	loan_category text
1	3094	Aarav More 1881	Personal Loan	112671.75	Small
2	1462	Meera Verma 299	Personal Loan	120717.00	Small
3	3372	Riya Sharma 1618	Personal Loan	123519.75	Small
4	2898	Aarav Gupta 3901	Personal Loan	127132.50	Small
5	3136	Meera Verma 1619	Personal Loan	128448.00	Small
6	3415	Meera Verma 1319	Personal Loan	129139.08	Small
7	2480	Arjun Verma 164	Personal Loan	130238.64	Small
8	2702	Aditya Nair 1883	Personal Loan	131712.00	Small
9	1504	Riya Nair 38	Personal Loan	136493.25	Small
Total rows: 5290		Query complete 00:00:00.749			

July 17 Section 2 — Date Functions & Analysis

Q6. Monthly Application Analysis

Using `TO_CHAR()`, `EXTRACT()` or `DATE_TRUNC()`, calculate monthly:

- Year
- Month
- Number of Applications
- Total Requested Amount

```

SELECT
EXTRACT(YEAR FROM application_date) AS Year,
TO_CHAR(application_date, 'MM') AS Month,
COUNT(loan_application_id) AS Total_Application,
SUM(requested_loan_amount) AS Total_Requested_Amount
FROM fact_loan_application
GROUP BY Year, Month
ORDER BY Year desc, Month desc;

```

Data Output Messages Notifications				
≡ + 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈 SQL Sh				
	year numeric 🔒	month text 🔒	total_application bigint 🔒	total_requested_amount numeric 🔒
1	2025	12	222	526811039.00
2	2025	11	215	486459793.00
3	2025	10	226	525383378.00
4	2025	09	221	537096036.00
5	2025	08	227	523260848.00
6	2025	07	229	544548893.00
7	2025	06	217	489299852.00
8	2025	05	229	541417340.00
9	2025	04	214	504741547.00
Total rows: 36		Query complete 00:00:00.189		

Q7. Loan Processing Time

Calculate the number of days between:

`application_date` → `approval_date`

Classify processing time using `CASE` :

- ≤ 3 days → Fast
- 4–7 days → Normal
- 8–15 days → Delayed
- 15 days → Highly Delayed

```
select loan_application_id , application_date, approval_date ,
nullif(approval_date - application_date,0) AS Processing_day,
(CASE
    WHEN approval_date IS NULL THEN 'Pending'
    WHEN approval_date - application_date BETWEEN 0 AND 3 TH
```

```

EN 'Fast'
    WHEN approval_date - application_date BETWEEN 4 AND 7 TH
EN 'Normal'
    WHEN approval_date - application_date BETWEEN 8 AND 15 T
HEN 'Delay'
    ELSE 'Highly Delayed'
END) AS Processing_Catogary
from fact_loan_application
order by application_date desc ;

```

Data Output Messages Notifications					
Showing rows: 1 to 1000					
	loan_application_id [PK] integer	application_date date	approval_date date	processing_day integer	processing_catogary text
1	1882	2025-12-30	[null]	[null]	Pending
2	2359	2025-12-30	[null]	[null]	Pending
3	4309	2025-12-30	2026-01-12	13	Delay
4	5782	2025-12-30	[null]	[null]	Pending
5	5305	2025-12-30	2026-01-14	15	Delay
6	1405	2025-12-30	2026-01-06	7	Normal
7	3355	2025-12-30	2026-01-10	11	Delay
8	3832	2025-12-30	2026-01-11	12	Delay
9	6259	2025-12-30	2026-01-02	3	Fast
10	1814	2025-12-29	2026-01-08	10	Delay
11	4760	2025-12-29	[null]	[null]	Pending
Total rows: 8000		Query complete 00:00:00.397			

Q8. EMI Payment Delay Analysis

Using `due_date` and `payment_date` , classify EMI payments as:

- On Time
- Minor Delay
- Moderate Delay
- Severe Delay

Also display the calculated delay days.

```
SELECT
loan_account_id,
installment_number,
due_date,
payment_date,
-- Calculate delay days
CASE
WHEN payment_date > due_date
THEN payment_date - due_date
ELSE 0
END AS delay_days,
-- Classify payment delay
CASE
WHEN payment_date <= due_date
THEN 'On Time'

WHEN payment_date - due_date BETWEEN 1 AND 7
THEN 'Minor Delay'

WHEN payment_date - due_date BETWEEN 8 AND 30
THEN 'Moderate Delay'

WHEN payment_date - due_date > 30
THEN 'Severe Delay'

ELSE 'Unknown'
END AS payment_delay_category

FROM fact_emi_payment
ORDER BY due_date;
```

Data Output

Messages

Notifications

									SQL	Showing rows: 1 to 1000					Pa
	loan_account_id integer	installment_number integer	due_date date	payment_date date	delay_days integer	payment_delay_category text									
1	3608	1	2023-02-09	2023-02-09	0	On Time									
2	3034	1	2023-02-13	2023-02-13	0	On Time									
3	2788	1	2023-02-13	2023-02-13	0	On Time									
4	1182	1	2023-02-13	2023-03-24	39	Severe Delay									
5	3685	1	2023-02-14	2023-02-14	0	On Time									
6	2216	1	2023-02-16	2023-02-16	0	On Time									
7	1959	1	2023-02-16	2023-02-16	0	On Time									
8	692	1	2023-02-16	2023-03-17	29	Moderate Delay									
9	4064	1	2023-02-16	2023-02-16	0	On Time									
10	4166	1	2023-02-16	2023-02-16	0	On Time									
Total rows: 157477		Query complete 00:00:01.267													

Q9. Year-wise Recovery Analysis

Calculate for every year:

- Recovery Cases
- Total Recovery Amount
- Average Recovery Amount
- Settlement Cases
- Written-off Cases

```
SELECT EXTRACT(YEAR FROM recovery_date) AS Year, COUNT(recovery_id) AS Recovery_Cases,
SUM(recovery_amount) AS Total_Recovery_Amount,
round(AVG(recovery_amount),2) AS Average_Recovery_Amount ,
count(CASE
WHEN settlement_flag='true' THEN settlement_flag
END) AS settlement_case ,
count(CASE
WHEN written_off_flag='true' THEN written_off_flag
END) AS written off case
```

```

from fact_recovery
GROUP BY Year
order by year desc;

```

Data Output Messages Notifications						
	year numeric	recovery_cases bigint	total_recovery_amount numeric	average_recovery_amount numeric	settlement_case bigint	written_off_case bigint
1	2026	55	4983965.37	90617.55	0	3
2	2025	218	21322863.37	97811.30	0	5
3	2024	89	8027832.92	90200.37	0	1

Total rows: 3 Query complete 00:00:00.415

Section 3 — CTE

Q10. Customer Loan Exposure

Create a CTE that calculates for every customer:

- Number of Loans
- Total Sanctioned Amount
- Total Outstanding Principal

Return customers whose **outstanding principal > ₹10 Lakh**.

```

WITH customer_exposure AS (
    SELECT
        la.customer_id,
        COUNT(loa.loan_account_id) AS number_of_loans,
        SUM(loa.sanctioned_amount) AS total_sanctioned_amount,
        SUM(loa.outstanding_principal) AS total_outstanding_principal
    FROM fact_loan_application la
    INNER JOIN fact_loan_account loa

```

```

        ON la.loan_application_id = loa.loan_application_id
    GROUP BY la.customer_id
)
SELECT
    c.customer_name,
    ce.number_of_loans,
    ce.total_sanctioned_amount,
    ce.total_outstanding_principal
FROM customer_exposure ce
INNER JOIN dim_customer c
    ON ce.customer_id = c.customer_id
WHERE ce.total_outstanding_principal > 1000000
order by ce.total_outstanding_principal asc;

```

Data Output Messages Notifications				
+ 📄 ▼ 📋 ▼ 🗑️ 🔒 📥 📈 SQL Showing rows: 1 to 1000				
	customer_name character varying (120)	number_of_loans bigint	total_sanctioned_amount numeric	total_outstanding_principal numeric
1	Aditya Joshi 2643	2	1550211.90	1000328.23
2	Riya Sharma 2578	1	1119730.50	1001156.37
3	Rohan Khan 3345	2	1413033.80	1001734.86
4	Aditi Verma 1529	1	1504877.40	1001751.11
5	Shreya Shah 860	2	2218065.36	1001767.92
6	Priya Patil 2711	1	1569470.40	1004572.76
7	Isha Patil 2516	1	1761425.40	1004989.48
8	Kunal Deshmukh 3927	1	1516027.50	1005288.21
9	Priya Gupta 2971	2	3214840.85	1006636.68
10	Isha Patil 1856	2	5886896.40	1007232.64
11	Aditya Joshi 723	1	1429556.40	1009282.09
12	Priya More 3831	1	1392304.80	1009435.70
13	Aditya Nair 1763	2	1652812.20	1010216.80
Total rows: 1960 Query complete 00:00:00.474				

Q11. High-Risk Loan Portfolio

Using a CTE:

1. Identify High and Very High risk applications.
2. Join them with loan accounts.
3. Calculate:
 - Number of Loans
 - Sanctioned Amount
 - Outstanding Principal

Show the results by **Loan Product**.

```
select application_status,risk_category from fact_loan_application;
```

```
WITH high_risk_applications AS (  
    SELECT  
        loan_application_id,  
        loan_product_id,  
        risk_category  
    FROM fact_loan_application  
    WHERE risk_category IN ('High', 'Very High')  
)
```

```
SELECT  
    lp.loan_product_name,  
    COUNT(la.loan_account_id) AS number_of_loans,  
    SUM(la.sanctioned_amount) AS sanctioned_amount,  
    SUM(la.outstanding_principal) AS outstanding_principal,  
    hra.risk_category  
FROM high_risk_applications hra  
  
INNER JOIN fact_loan_account la  
    ON hra.loan_application_id = la.loan_application_id  
  
INNER JOIN dim_loan_product lp  
    ON hra.loan_product_id = lp.loan_product_id
```

GROUP BY

```
lp.loan_product_id,  
lp.loan_product_name,  
hra.risk_category
```

ORDER BY

```
outstanding_principal DESC;
```

Data Output Messages Notifications					
Showing rows: 1 to 5					
	loan_product_name character varying (100)	number_of_loans bigint	sanctioned_amount numeric	outstanding_principal numeric	risk_category character varying (40)
1	Home Loan	329	1530047726.76	1407303221.24	High
2	Home Loan	59	256688079.75	237174828.71	Very High
3	Personal Loan	514	463832657.40	204862425.17	High
4	Vehicle Loan	282	320034871.80	197821567.13	High
5	Personal Loan	204	158174100.00	75312923.28	Very High
Total rows: 5 Query complete 00:00:00.155					

Q12. Delinquency Analysis

Create a CTE to calculate:

- DPD Bucket
- Number of Loans
- Outstanding Principal
- Overdue Amount

Also calculate each bucket's **percentage contribution to total delinquent outstanding**.

```
WITH delinquency_data AS (  
    SELECT
```

```

        lac.dpd_bucket,
        COUNT(DISTINCT lac.loan_account_id) AS number_of_loans,
        SUM(lac.outstanding_principal) AS outstanding_principal,
        SUM(ds.overdue_amount) AS overdue_amount
FROM fact_loan_account lac
    INNER JOIN fact_delinquency_snapshot ds
    ON lac.loan_account_id = ds.loan_account_id
GROUP BY lac.dpd_bucket
)

SELECT
    dpd_bucket,
    number_of_loans,
    outstanding_principal,
    overdue_amount,
    ROUND(
        outstanding_principal * 100.0 /
        NULLIF(SUM(outstanding_principal) OVER (), 0),
        2
    ) AS percentage_contribution
FROM delinquency_data
ORDER BY outstanding_principal DESC;

```

Data Output Messages Notifications					
+ SQL 📄 📋 🗑️ 📊 📥 📈 Showing rows: 1 to 4 🔍 📄 Page No:					
	dpd_bucket character varying (40)	number_of_loans bigint	outstanding_principal numeric	overdue_amount numeric	percentage_contribution numeric
1	31-60 DPD	365	1963504772.98	86550535.70	53.16
2	1-30 DPD	281	992235208.25	20927885.88	26.86
3	61-90 DPD	116	656632236.99	40781939.64	17.78
4	90+ DPD	16	81066073.64	7205249.12	2.19

Total rows: 4

Query complete 00:00:00.460

Q13. Repeat Applicants

Using a CTE, identify customers who have submitted **more than one loan application**.

Display:

- Customer Name
- Total Applications
- Approved Applications
- Rejected Applications
- Total Requested Amount

```
WITH repeat_applicants AS (  
    SELECT  
        customer_id,  
        COUNT(loan_application_id) AS total_applications,  
        COUNT(CASE  
            WHEN application_status = 'Approved'  
            THEN loan_application_id  
        END) AS approved_applications,  
        COUNT(CASE  
            WHEN application_status = 'Rejected'  
            THEN loan_application_id  
        END) AS rejected_applications,  
        SUM(requested_loan_amount) AS total_requested_amount  
    FROM fact_loan_application  
    GROUP BY customer_id  
    HAVING COUNT(loan_application_id) > 1  
)  
  
SELECT  
    c.customer_name,  
    ra.total_applications,  
    ra.approved_applications,  
    ra.rejected_applications,
```

```

    ra.total_requested_amount
FROM repeat_applicants ra
INNER JOIN dim_customer c
    ON ra.customer_id = c.customer_id
ORDER BY ra.total_applications DESC;

```

Data Output Messages Notifications						
	customer_name character varying (120)	total_applications bigint	approved_applications bigint	rejected_applications bigint	total_requested_amount numeric	
1	Riya Sharma 1798	3	2	1	3529419.00	
2	Aditya Joshi 1203	3	1	2	2246621.00	
3	Aarav Gupta 1801	3	3	0	4229717.00	
4	Sneha Mehta 2692	3	3	0	4890039.00	
5	Rohan Shah 1205	3	0	1	5583505.00	
6	Aditya Joshi 1803	3	3	0	12102001.00	
7	Riya Joshi 798	3	1	2	1593162.00	
Total rows: 3013 Query complete 00:00:00.317						

Q14. Monthly Collection Performance

Using a CTE, calculate monthly:

- Total Amount Demanded
- Total Amount Collected
- Collection Efficiency %

Identify the month with the **highest collection efficiency**.

```

WITH monthly_collection AS (
    SELECT
        EXTRACT(YEAR FROM activity_date) AS Year,
        EXTRACT(MONTH FROM activity_date) AS Month,
        SUM(amount_demanded) AS total_amount_demanded,
        SUM(amount_collected) AS total_amount_collected
    FROM fact_collection_activity

```

```

GROUP BY Year,Month
)

SELECT
    Year,
    month,
    total_amount_demanded,
    total_amount_collected,
    ROUND(
        total_amount_collected * 100.0 /
        NULLIF(total_amount_demanded, 0),
        2
    ) AS collection_efficiency
FROM monthly_collection
ORDER BY Year desc,Month asc;

```

Data Output Messages Notifications						
	year numeric	month numeric	total_amount_demanded numeric	total_amount_collected numeric	collection_efficiency numeric	
1	2026	1	6730300.28	4165720.28	61.90	
2	2026	2	6674373.10	4242674.03	63.57	
3	2026	3	6955976.77	4173995.18	60.01	
4	2026	4	6065829.72	4418724.56	72.85	
5	2026	5	7289179.40	5021802.75	68.89	
6	2026	6	6843298.78	4705506.66	68.76	
7	2026	7	6982996.40	4167830.94		

Total rows: 30 Query complete 00:00:00.264

Showing rows: 1 to 30 Page No: 1 of 1

Successfully run. Total query runtime: 253 msec. 30 rows affected.

Section 4 — Views

Q15. Customer Loan Summary View

Create a view:

```
vw_customer_loan_summary
```

It should contain:

- Customer ID

- Customer Name
- Customer Segment
- Loan Count
- Total Sanctioned Amount
- Total Outstanding Principal
- Delinquent Loan Count

```
CREATE OR REPLACE VIEW vw_customer_loan_summary AS

WITH loan_summary AS (
    SELECT
        la.customer_id,
        COUNT(DISTINCT loa.loan_account_id) AS loan_count,
        SUM(loa.sanctioned_amount) AS total_sanctioned_amount,
        SUM(loa.outstanding_principal) AS total_outstanding_principal
    FROM fact_loan_application la
    INNER JOIN fact_loan_account loa
        ON la.loan_application_id = loa.loan_application_id
    GROUP BY la.customer_id
),

delinquency_summary AS (
    SELECT
        la.customer_id,
        COUNT(DISTINCT ds.loan_account_id) AS delinquent_loan_count
    FROM fact_loan_application la
    INNER JOIN fact_loan_account loa
        ON la.loan_application_id = loa.loan_application_id
    INNER JOIN fact_delinquency_snapshot ds
        ON loa.loan_account_id = ds.loan_account_id
```

```

WHERE ds.dpd_days > 0
GROUP BY la.customer_id
)

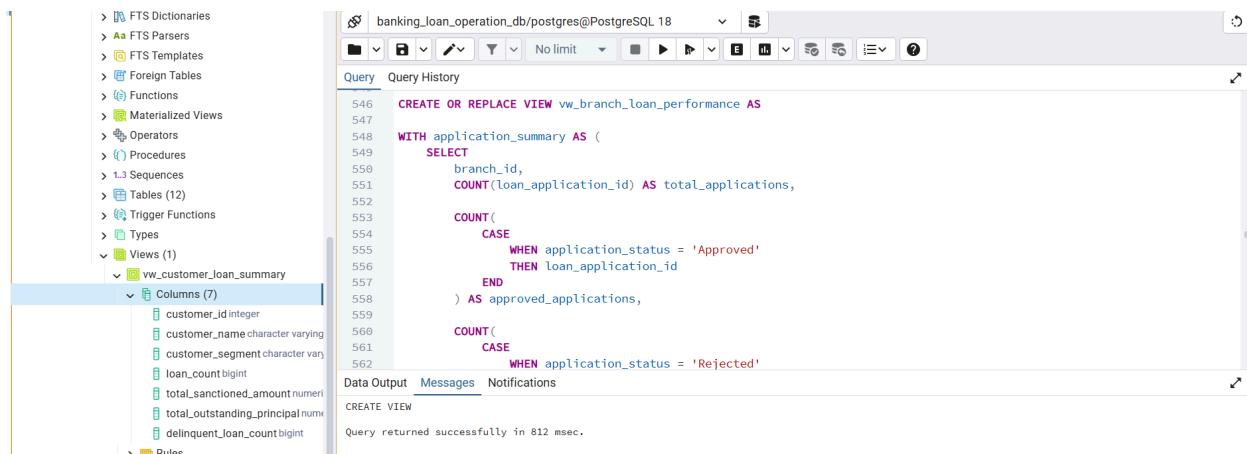
SELECT
    c.customer_id,
    c.customer_name,
    c.customer_segment,
    ls.loan_count,
    ls.total_sanctioned_amount,
    ls.total_outstanding_principal,
    COALESCE(ds.delinquent_loan_count, 0) AS delinquent_loan_
count

FROM dim_customer c

INNER JOIN loan_summary ls
    ON c.customer_id = ls.customer_id

LEFT JOIN delinquency_summary ds
    ON c.customer_id = ds.customer_id;

```



```

SELECT *

```

```
FROM vw_customer_loan_summary;
```

Data Output Messages Notifications								
Showing rows: 1 to 1000 Page No: 1 of 4								
	customer_id integer	customer_name character varying (120)	customer_segment character varying (60)	loan_count bigint	total_sanctioned_amount numeric	total_outstanding_principal numeric	delinquent_loan_count bigint	
1	1	Aarav Gupta 1	Mass Market	2	7379742.75	6720930.73	2	
2	3	Aditya Joshi 3	Affluent	3	9678906.90	7501722.07	0	
3	4	Arjun Singh 4	Affluent	1	3130399.80	1728308.82	1	
4	5	Rohan Shah 5	Affluent	2	2033020.80	1073722.97	0	
5	6	Rahul More 6	Affluent	1	6669101.70	5826103.89	0	
6	8	Siddharth Nair 8	Mass Affluent	3	3449258.10	1796595.61	1	
7	9	Aditi Kulkarni 9	Mass Affluent	1	484932.25	179730.79	0	
Total rows: 3374 Query complete 00:00:00.530 CRLF Ln 526, Cc								

Q16. Branch Performance View

Create:

```
vw_branch_loan_performance
```

Include:

- Region
- Branch
- Total Applications
- Approved Applications
- Rejected Applications
- Approval Rate
- Total Sanctioned Amount

```
CREATE OR REPLACE VIEW vw_branch_loan_performance AS
```

```
WITH application_summary AS (
    SELECT
        branch_id,
        COUNT(loan_application_id) AS total_applications,
```

```

COUNT(
    CASE
        WHEN application_status = 'Approved'
        THEN loan_application_id
    END
) AS approved_applications,

COUNT(
    CASE
        WHEN application_status = 'Rejected'
        THEN loan_application_id
    END
) AS rejected_applications
FROM fact_loan_application
GROUP BY branch_id
),

sanction_summary AS (
    SELECT
        la.branch_id,
        SUM(loa.sanctioned_amount) AS total_sanctioned_amount
    FROM fact_loan_application la
    INNER JOIN fact_loan_account loa
        ON la.loan_application_id = loa.loan_application_id
    GROUP BY la.branch_id
)

SELECT

    b.branch_name AS branch,

    COALESCE(a.total_applications, 0) AS total_applications,

    COALESCE(a.approved_applications, 0) AS approved_applications,

```

```

        COALESCE(a.rejected_applications, 0) AS rejected_applications,

        ROUND(
            COALESCE(a.approved_applications, 0) * 100.0
            / NULLIF(COALESCE(a.total_applications, 0), 0),
            2
        ) AS approval_rate,

        COALESCE(s.total_sanctioned_amount, 0) AS total_sanctioned_amount

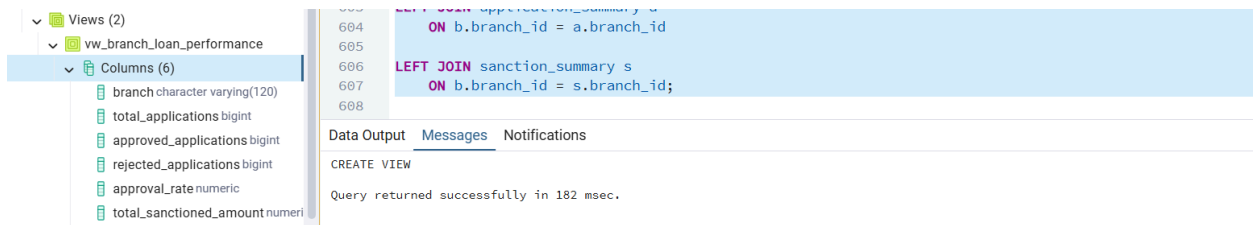
FROM dim_region r

INNER JOIN dim_branch b
    ON r.region_id = b.region_id

LEFT JOIN application_summary a
    ON b.branch_id = a.branch_id

LEFT JOIN sanction_summary s
    ON b.branch_id = s.branch_id;

```



```

SELECT *
FROM vw_branch_loan_performance;

```

Data Output Messages Notifications							
Showing rows: 1 to 45 of Page No: 1							
	branch character varying (120)	total_applications bigint	approved_applications bigint	rejected_applications bigint	approval_rate numeric	total_sanctioned_amount numeric	
1	Mumbai Andheri Branch	178	129	41	72.47	321805270.45	
2	Mumbai Borivali Branch	175	124	45	70.86	286695234.23	
3	Mumbai Thane Branch	176	119	53	67.61	285511193.52	
4	Mumbai Dadar Branch	180	116	59	64.44	310164087.33	
5	Mumbai Navi Mumbai Branch	178	111	62	62.36	263529839.71	
6	Mumbai Powai Branch	175	107	59	61.14	266562804.17	
7	Pune Camp Branch	174	116	53	66.67	226932118.82	
Total rows: 45 Query complete 00:00:00.165 CR							

Q17. Loan Delinquency View

Create:

`vw_loan_delinquency`

Include:

- Customer
- Loan Account
- Loan Product
- Outstanding Principal
- DPD Bucket
- Overdue Amount
- Collection Amount
- Recovery Amount

```
CREATE OR REPLACE VIEW vw_loan_delinquency AS

WITH delinquency_data AS (
    SELECT DISTINCT ON (loan_account_id)
        loan_account_id,
        dpd_bucket,
        overdue_amount
    FROM fact_delinquency_snapshot
    ORDER BY loan_account_id, snapshot_date DESC
```

```

),

collection_data AS (
    SELECT
        loan_account_id,
        SUM(amount_collected) AS collection_amount
    FROM fact_collection_activity
    GROUP BY loan_account_id
),

recovery_data AS (
    SELECT
        loan_account_id,
        SUM(recovery_amount) AS recovery_amount
    FROM fact_recovery
    GROUP BY loan_account_id
)

SELECT
    c.customer_name AS customer,
    loa.loan_account_number AS loan_account,
    lp.loan_product_name AS loan_product,
    loa.outstanding_principal,
    dd.dpd_bucket,
    dd.overdue_amount,
    COALESCE(cd.collection_amount, 0) AS collection_amount,
    COALESCE(rd.recovery_amount, 0) AS recovery_amount

FROM fact_loan_account loa

INNER JOIN fact_loan_application la
    ON loa.loan_application_id = la.loan_application_id

INNER JOIN dim_customer c
    ON la.customer_id = c.customer_id

```

```

INNER JOIN dim_loan_product lp
    ON la.loan_product_id = lp.loan_product_id

LEFT JOIN delinquency_data dd
    ON loa.loan_account_id = dd.loan_account_id

LEFT JOIN collection_data cd
    ON loa.loan_account_id = cd.loan_account_id

LEFT JOIN recovery_data rd
    ON loa.loan_account_id = rd.loan_account_id;

```

Views (3)

- vw_branch_loan_performance
- vw_customer_loan_summary
- vw_loan_delinquency
 - Columns (8)
 - customer character varying(120)
 - loan_account character varying(50)
 - loan_product character varying(100)
 - outstanding_principal numeric(14,2)
 - dpd_bucket character varying(40)
 - overdue_amount numeric(14,2)
 - collection_amount numeric
 - recovery_amount numeric

```

685     ON loa.loan_account_id = rd.loan_account_id;
686
687 SELECT *
688 FROM vw_loan_delinquency;
689

```

Data Output Messages Notifications

CREATE VIEW

Query returned successfully in 214 msec.

```

SELECT *
FROM vw_loan_delinquency;

```

Data Output Messages Notifications									
Showing rows: 1 to 1000 Page No: 1 of 6									
	customer character varying (120)	loan_account character varying (50)	loan_product character varying (100)	outstanding_principal numeric (14,2)	dpd_bucket character varying (40)	overdue_amount numeric (14,2)	collection_amount numeric	recovery_amount numeric	
1	Kavya Rao 3535	LN202300000743	Home Loan	3858967.10	[null]	[null]	0		
2	Aarav Patil 3461	LN202600000741	Home Loan	3938877.58	[null]	[null]	0		
3	Arjun Singh 3424	LN202500000740	Home Loan	3582245.15	[null]	[null]	0		
4	Kunal Deshmukh 3387	LN202500000739	Home Loan	3733179.01	[null]	[null]	0		
5	Ananya Shah 3350	LN202400000738	Home Loan	3614004.92	[null]	[null]	0		
6	Neha Sharma 3313	LN202400000737	Home Loan	3498483.22	31-60 DPD	70088.26	158049.04		
Total rows: 5290 Query complete 00:00:00.279 CRLF Ln 687, Col 1									

Q18. View-Based Analysis

Using `vw_customer_loan_summary`, find the **Top 20 customers by outstanding principal**.

Do not directly query the underlying tables for this question.

```
SELECT
    customer_id,
    customer_name,
    customer_segment,
    loan_count,
    total_sanctioned_amount,
    total_outstanding_principal,
    delinquent_loan_count
FROM vw_customer_loan_summary
ORDER BY total_outstanding_principal DESC
LIMIT 20;
```

	customer_id integer	customer_name character varying (120)	customer_segment character varying (60)	loan_count bigint	total_sanctioned_amount numeric	total_outstanding_principal numeric	delinquent_loan_count bigint
1	1969	Aditi Singh 1969	Mass Affluent	3	19782025.20	17579365.90	0
2	2653	Neha Sharma 2653	Mass Affluent	3	18753680.20	16768286.02	1
3	2616	Isha More 2616	Premium	3	18555777.15	16663028.42	1
4	2285	Rohan Shah 2285	Affluent	3	17152893.00	15687296.70	0
5	675	Kavya Khan 675	Premium	3	16823642.15	15679154.04	0
6	2248	Siddharth Sharma 2248	Mass Affluent	3	16974411.30	15143871.60	0
7	638	Riya Nair 638	Mass Market	3	16567533.00	14367754.41	1
8	3165	Rohan Khan 3165	Affluent	3	16289712.70	14246548.92	0
9	2211	Priya More 2211	Mass Affluent	3	16795929.60	14227081.14	0
10	2391	Priya More 2391	Mass Affluent	3	15317358.30	14051790.06	0
11	973	Neha Sharma 973	Mass Affluent	3	17308066.15	13979422.03	0
12	3431	Priya Patil 3431	Mass Affluent	3	17112478.25	13930443.21	0
13	2428	Siddharth Sharma 2428	Mass Affluent	3	15477894.00	13871110.01	0
14	3505	Rohan Rao 3505	Affluent	3	16530516.90	13733506.54	1
15	487	Kunal Mehta 487	Mass Affluent	2	14364264.10	13653357.93	1
16	381	Aarav More 381	Mass Market	3	13986897.30	13482741.99	0
17	2505	Rohan Khan 2505	Affluent	3	14247759.60	13424828.14	1
18	2354	Pooja Verma 2354	Premium	2	14530168.30	13385162.39	0
19	2431	Priya Gupta 2431	Mass Affluent	3	14662507.10	13367347.23	0
20	3991	Priya Gupta 3991	Mass Affluent	2	14018435.10	13169985.65	0

Total rows: 20 Query complete 00:00:00.256 CRLF Ln 709, Col 7

Section 5 — PostgreSQL Functions

Q19. Customer Loan Exposure Function

Create:

```
fn_customer_loan_exposure(customer_id)
```

The function should return:

- Total Loans
- Total Sanctioned Amount
- Total Outstanding Principal

Test the function for at least **5 customers**.

```
CREATE OR REPLACE FUNCTION fn_customer_loan_exposure(
    p_customer_id INT
)
RETURNS TABLE (
    total_loans BIGINT,
    total_sanctioned_amount NUMERIC,
    total_outstanding_principal NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN

    RETURN QUERY
    SELECT
        COUNT(loa.loan_account_id),
        COALESCE(SUM(loa.sanctioned_amount), 0),
        COALESCE(SUM(loa.outstanding_principal), 0)

    FROM fact_loan_application la
```

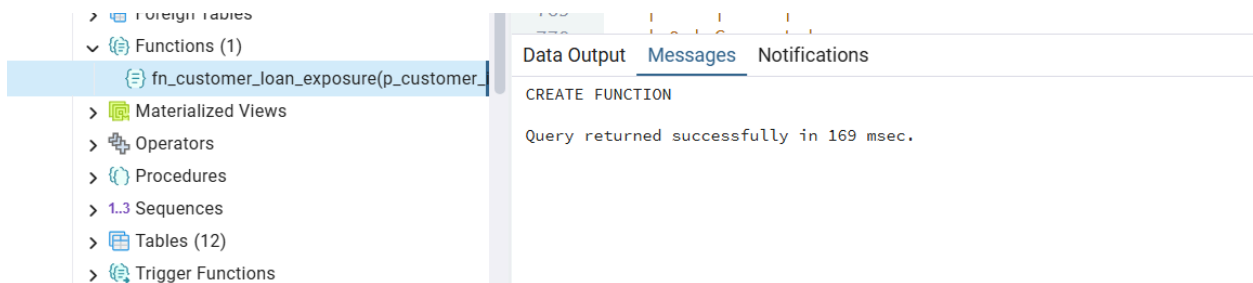
```

INNER JOIN fact_loan_account loa
  ON la.loan_application_id = loa.loan_application_id

WHERE la.customer_id = p_customer_id;

END;
$$;

```



```

SELECT *
FROM fn_customer_loan_exposure(1);

```

The screenshot shows a database management tool interface. The 'Data Output' tab is active, displaying a table with loan statistics. The table has four columns: 'total_loans' (bigint), 'total_sanctioned_amount' (numeric), and 'total_outstanding_principal' (numeric). The first row shows values for these metrics.

	total_loans bigint	total_sanctioned_amount numeric	total_outstanding_principal numeric
1	2	7379742.75	6720930.73

Q20. DPD Classification Function

Create:

```
fn_dpd_bucket(dpd_days)
```

The function should return:

DPD	Bucket
0	Current
1-30	1-30 DPD
31-60	31-60 DPD
61-90	61-90 DPD
90+	90+ DPD

Test it with different DPD values.

```
CREATE OR REPLACE FUNCTION fn_dpd_bucket(dpd_days INT)
RETURNS VARCHAR
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN CASE
        WHEN dpd_days = 0 THEN 'Current'
        WHEN dpd_days BETWEEN 1 AND 30 THEN '1-30 DPD'
        WHEN dpd_days BETWEEN 31 AND 60 THEN '31-60 DPD'
        WHEN dpd_days BETWEEN 61 AND 90 THEN '61-90 DPD'
        WHEN dpd_days > 90 THEN '90+ DPD'
        ELSE 'Invalid DPD'
    END;
END;
$$;
```

The screenshot shows a database IDE interface. On the left, a tree view under 'Functions (2)' lists 'fn_dpd_bucket(dpd_days integer)' as the selected item. On the right, the 'Messages' tab displays the message: 'CREATE FUNCTION' followed by 'Query returned successfully in 187 msec.'

```
SELECT fn_dpd_bucket(0);
```

Data Output

Messages

Notifications

≡+

▼

▼

SQL

	fn_dpd_bucket character varying
1	Current

Q21. Loan EMI Function

Create:

```
fn_calculate_emi(loan_amount, annual_interest_rate, tenure_months)
```

The function should calculate and return the **monthly EMI**.

Test it using at least **3 different loan scenarios**.

```
CREATE OR REPLACE FUNCTION fn_calculate_emi(
    loan_amount NUMERIC,
    annual_interest_rate NUMERIC,
    tenure_months INTEGER
)
RETURNS NUMERIC
LANGUAGE plpgsql
AS $$
DECLARE
    monthly_rate NUMERIC;
    emi NUMERIC;
BEGIN
    -- Validation
    IF loan_amount <= 0 THEN
        RAISE EXCEPTION 'Loan amount must be greater than 0';
    END IF;

    IF annual_interest_rate < 0 THEN
```

```

        RAISE EXCEPTION 'Interest rate cannot be negative';
    END IF;

    IF tenure_months <= 0 THEN
        RAISE EXCEPTION 'Tenure must be greater than 0';
    END IF;

    -- Convert annual interest rate to monthly rate
    monthly_rate := annual_interest_rate / 12 / 100;

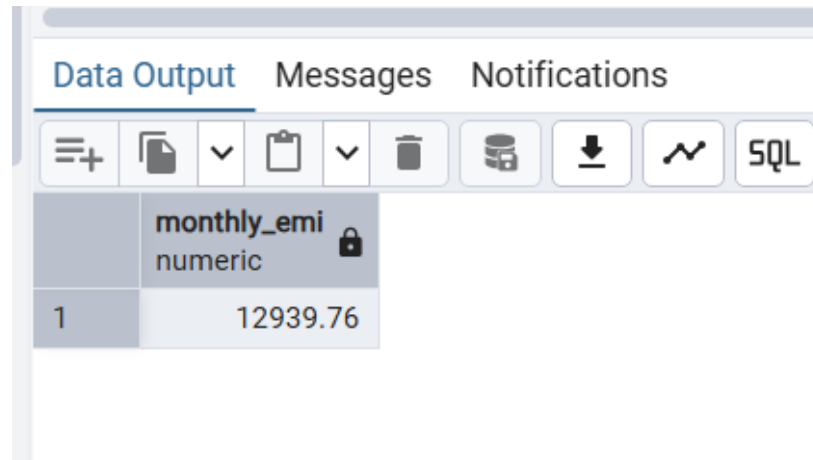
    -- EMI calculation
    IF monthly_rate = 0 THEN
        emi := loan_amount / tenure_months;
    ELSE
        emi := (
            loan_amount
            * monthly_rate
            * POWER(1 + monthly_rate, tenure_months)
        )
        / (
            POWER(1 + monthly_rate, tenure_months) - 1
        );
    END IF;

    RETURN ROUND(emi, 2);
END;
$$;

```

The screenshot shows a database IDE interface. On the left, a tree view displays the database structure, including 'Foreign Tables', 'Functions (3)', 'Materialized Views', 'Operators', and 'Procedures'. The 'Functions (3)' folder is expanded, and the function 'fn_dpd_bucket(dpd_days integer)' is selected. The main editor area shows the SQL code for the function, with line numbers 848 through 851 visible. The 'Messages' tab is active, showing the output of the query: 'CREATE FUNCTION' and 'Query returned successfully in 165 msec.'

```
SELECT fn_calculate_emi(1000000, 9.5, 120) AS monthly_emi;
```



	monthly_emi numeric
1	12939.76

Section 6 — Stored Procedures

Q22. Update Loan DPD Procedure

Create:

```
sp_update_dpd_bucket()
```

The procedure should update `fact_loan_account.dpd_bucket` based on the latest available delinquency information.

```
CREATE OR REPLACE PROCEDURE sp_update_dpd_bucket()  
LANGUAGE plpgsql  
AS $$  
BEGIN  
  
    UPDATE fact_loan_account fla  
    SET dpd_bucket =  
        CASE  
            WHEN emi.dpd_days = 0
```

```

        THEN 'Current'

    WHEN emi.dpd_days BETWEEN 1 AND 30
        THEN '1-30 DPD'

    WHEN emi.dpd_days BETWEEN 31 AND 60
        THEN '31-60 DPD'

    WHEN emi.dpd_days BETWEEN 61 AND 90
        THEN '61-90 DPD'

    WHEN emi.dpd_days > 90
        THEN '90+ DPD'

    ELSE 'Unknown'
END

FROM
(
    SELECT
        loan_account_id,
        CASE
            WHEN payment_date > due_date
            THEN payment_date - due_date
            ELSE 0
        END AS dpd_days
    FROM fact_emi_payment
) emi

WHERE fla.loan_account_id = emi.loan_account_id;

END;
$$;

```

Operators

Procedures (3)

- { } sp_process_emi_payment(IN p_loan_account_id integer, IN p_installment_number integer, IN p_payment_date datetime, IN p_amount_paid decimal, IN p_payment_mode varchar(50))
- { } sp_record_recovery(IN p_loan_account_id integer, IN p_installment_number integer, IN p_payment_date datetime, IN p_amount_paid decimal, IN p_payment_mode varchar(50))
- { } sp_update_dpd_bucket

1.3 Sequences

Tables (12)

Trigger Functions

Types

```

885
886 CALL sp_update_dpd_bucket();
887
888 select * from fact_loan_account;

```

Data Output Messages Notifications

CREATE PROCEDURE

Query returned successfully in 194 msec.

```
CALL sp_update_dpd_bucket();
```

Procedures (3)

- { } sp_process_emi_payment(IN p_loan_account_id integer, IN p_installment_number integer, IN p_payment_date datetime, IN p_amount_paid decimal, IN p_payment_mode varchar(50))
- { } sp_record_recovery(IN p_loan_account_id integer, IN p_installment_number integer, IN p_payment_date datetime, IN p_amount_paid decimal, IN p_payment_mode varchar(50))
- { } sp_update_dpd_bucket

1.3 Sequences

Tables (12)

Trigger Functions

Types

```

919 -- `sp_process_emi_payment()`
920
921 -- Parameters should include:
922

```

Data Output Messages Notifications

CALL

Query returned successfully in 911 msec.

Q23. Process EMI Payment Procedure

Create:

```
sp_process_emi_payment()
```

Parameters should include:

- Loan Account ID
- Installment Number
- Payment Date
- Amount Paid
- Payment Mode

The procedure should:

1. Locate the EMI record.
2. Update payment information.

3. Calculate delay days.
4. Determine payment status using `CASE`.

```
CREATE OR REPLACE PROCEDURE sp_process_emi_payment(  
    p_loan_account_id INT,  
    p_installment_number INT,  
    p_payment_date DATE,  
    p_amount_paid NUMERIC(15,2),  
    p_payment_mode VARCHAR(50)  
)  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_due_date DATE;  
BEGIN  
  
    -- 1. Locate the EMI record  
    SELECT due_date  
    INTO v_due_date  
    FROM fact_emi_payment  
    WHERE loan_account_id = p_loan_account_id  
        AND installment_number = p_installment_number;  
  
    -- Check whether EMI exists  
    IF NOT FOUND THEN  
        RAISE EXCEPTION  
        'EMI record not found for Loan Account ID % and Installment %',  
        p_loan_account_id, p_installment_number;  
    END IF;  
  
    -- 2, 3 & 4. Update payment information,  
    -- calculate delay days and determine payment status  
    UPDATE fact_emi_payment  
    SET
```

```

payment_date = p_payment_date,
amount_paid = p_amount_paid,
payment_mode = p_payment_mode,

delay_days =
CASE
    WHEN p_payment_date > v_due_date
    THEN p_payment_date - v_due_date
    ELSE 0
END,

payment_status =
CASE
    WHEN p_payment_date <= v_due_date
    THEN 'On Time'
    WHEN p_payment_date > v_due_date
    AND p_payment_date <= v_due_date + 30
    THEN 'Late'
    WHEN p_payment_date > v_due_date + 30
    THEN 'Severely Late'
    ELSE 'Unknown'
END

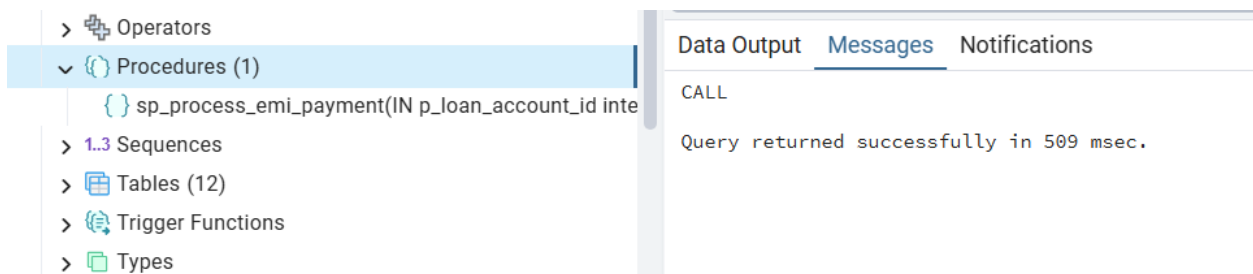
WHERE loan_account_id = p_loan_account_id
    AND installment_number = p_installment_number;

END;
$$;

```

	Data Output	Messages	Notifications
> Operators ✓ Procedures (1) { } sp_process_emi_payment(IN p_loan_account_id inte > 1.3 Sequences > Tables (12) > Trigger Functions	<pre>CREATE PROCEDURE</pre> Query returned successfully in 131 msec.		

```
CALL sp_process_emi_payment (
    1001,
    3,
    '2026-08-20',
    12500.00,
    'UPI'
);
```



Q24. Record Loan Recovery Procedure

Create:

```
sp_record_recovery()
```

Parameters:

- Loan Account ID
- Agent ID
- Recovery Date
- Recovery Amount
- Recovery Type

The procedure should validate that the loan is **delinquent/problematic** before inserting the recovery record.

```
CREATE OR REPLACE PROCEDURE sp_record_recovery(
    p_loan_account_id INT,
    p_agent_id INT,
    p_recovery_date DATE,
    p_recovery_amount NUMERIC(15,2),
```

```

        p_recovery_type VARCHAR(50)
    )
LANGUAGE plpgsql
AS $$
DECLARE
    v_dpd_bucket VARCHAR(50);
BEGIN

    -- 1. Locate the loan and get DPD bucket
    SELECT dpd_bucket
    INTO v_dpd_bucket
    FROM fact_loan_account
    WHERE loan_account_id = p_loan_account_id;

    -- Loan not found
    IF NOT FOUND THEN
        RAISE EXCEPTION
            'Loan Account ID % does not exist',
            p_loan_account_id;
    END IF;

    -- 2. Validate delinquent/problematic loan
    IF v_dpd_bucket = 'Current' THEN
        RAISE EXCEPTION
            'Recovery cannot be recorded. Loan Account ID % is Current.',
            p_loan_account_id;
    END IF;

    -- 3. Validate recovery amount
    IF p_recovery_amount <= 0 THEN
        RAISE EXCEPTION
            'Recovery amount must be greater than zero.';
    END IF;

    -- 4. Insert recovery record

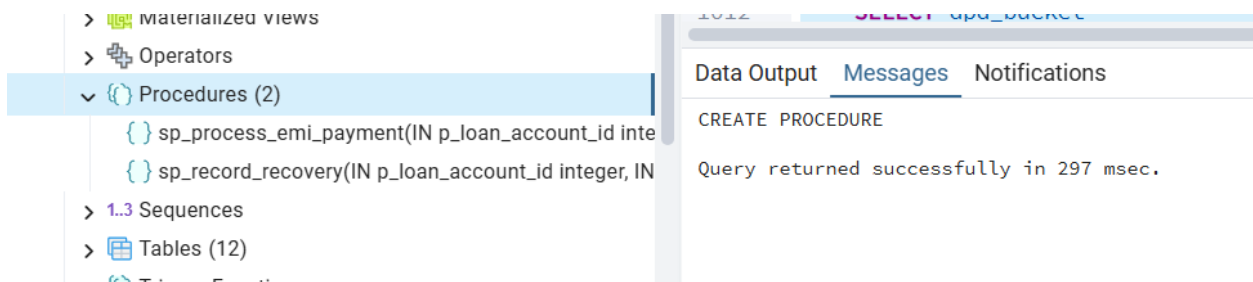
```

```

INSERT INTO fact_loan_recovery (
    loan_account_id,
    agent_id,
    recovery_date,
    recovery_amount,
    recovery_type
)
VALUES (
    p_loan_account_id,
    p_agent_id,
    p_recovery_date,
    p_recovery_amount,
    p_recovery_type
);

END;
$$;

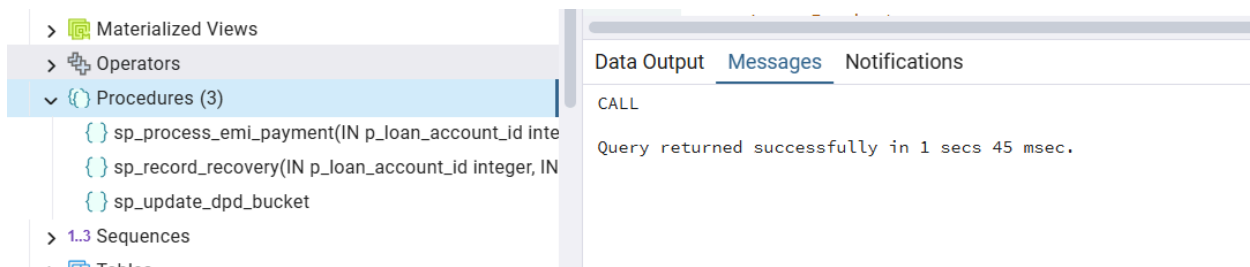
```



```

CALL sp_record_recovery(
    1302,
    25,
    '2026-08-21',
    15000.00,
    'Cash'
);

```



Section 7 — Final Business Challenge

Q25. Executive Loan Portfolio Report

Create one final management-level SQL report containing:

- Loan Product
- Total Applications
- Approval Rate
- Total Loans
- Total Sanctioned Amount
- Total Outstanding Principal
- Delinquent Loans
- Delinquency Rate
- Total Recovery
- Recovery Rate

Requirements

Your solution **must use**:

-  Multiple JOINS
-  At least 2 CTEs

-  CASE
-  Date functions
-  Aggregate functions
-  Meaningful column aliases

Expected Outcome

The final query should answer:

"What is the current performance and risk position of each loan product?"

```
WITH application_summary AS (
    SELECT
        loan_product_id,

        COUNT(loan_application_id) AS total_applications,

        ROUND(
            100.0 *
            COUNT(
                CASE
                    WHEN application_status = 'Approved'
                    THEN 1
                END
            )
            / NULLIF(COUNT(loan_application_id), 0),
            2
        ) AS approval_rate

    FROM fact_loan_application

    GROUP BY loan_product_id
),

loan_summary AS (
```

```

SELECT
    la.loan_product_id,

    COUNT(DISTINCT loa.loan_account_id) AS total_loans,

    SUM(loa.sanctioned_amount) AS total_sanctioned_amount,

    SUM(loa.outstanding_principal) AS total_outstanding_principal,

    COUNT(
        CASE
            WHEN ds.dpd_days > 0
            THEN loa.loan_account_id
        END
    ) AS delinquent_loans,

    SUM(
        CASE
            WHEN ds.dpd_days > 0
            THEN ds.overdue_amount
            ELSE 0
        END
    ) AS total_overdue_amount,

    SUM(r.total_recovery) AS total_recovery

FROM fact_loan_application la

INNER JOIN fact_loan_account loa
    ON la.loan_application_id = loa.loan_application_id

LEFT JOIN (
    SELECT
        loan_account_id,

```

```

        dpd_days,
        overdue_amount
FROM (
    SELECT
        loan_account_id,
        dpd_days,
        overdue_amount,
        ROW_NUMBER() OVER (
            PARTITION BY loan_account_id
            ORDER BY snapshot_date DESC
        ) AS rn
    FROM fact_delinquency_snapshot
) d
WHERE rn = 1
) ds
ON loa.loan_account_id = ds.loan_account_id

LEFT JOIN (
    SELECT
        loan_account_id,
        SUM(recovery_amount) AS total_recovery
    FROM fact_recovery
    WHERE DATE_TRUNC('year', recovery_date)
        = DATE_TRUNC('year', CURRENT_DATE)
    GROUP BY loan_account_id
) r
ON loa.loan_account_id = r.loan_account_id

GROUP BY la.loan_product_id
)

SELECT
    lp.loan_product_name AS "Loan Product",

    a.total_applications AS "Total Applications",

```

```

a.approval_rate AS "Approval Rate (%)",

l.total_loans AS "Total Loans",

l.total_sanctioned_amount AS "Total Sanctioned Amount",

l.total_outstanding_principal AS "Total Outstanding Principal",

l.delinquent_loans AS "Delinquent Loans",

ROUND(
    100.0 * l.delinquent_loans
    / NULLIF(l.total_loans, 0),
    2
) AS "Delinquency Rate (%)",

l.total_recovery AS "Total Recovery",

ROUND(
    100.0 * l.total_recovery
    / NULLIF(l.total_overdue_amount, 0),
    2
) AS "Recovery Rate (%)"

FROM dim_loan_product lp

INNER JOIN application_summary a
    ON lp.loan_product_id = a.loan_product_id

INNER JOIN loan_summary l
    ON lp.loan_product_id = l.loan_product_id

ORDER BY l.total_outstanding_principal DESC;

```

Data Output

Messages

Notifications

Showing rows: 1 to 5

Page No: 1 of 1

	Loan Product character varying (100)	Total Applications bigint	Approval Rate (%) numeric	Total Loans bigint	Total Sanctioned Amount numeric	Total Outstanding Principal numeric	Delinquent Loans bigint	Delinquency Rate (%) numeric	Total Recovery numeric	Reco nume
1	Home Loan	1360	94.78	1289	6248692712.71	5745923098.42	210	16.29	1430730.73	
2	Business Loan	880	67.50	594	1936446945.05	1195164479.74	75	12.63	766424.27	
3	Personal Loan	2160	100.00	2160	2052182856.40	908449006.67	310	14.35	2102866.53	
4	Mortgage Loan	720	32.36	233	850053906.00	787141965.69	32	13.73	91222.92	
5	Vehicle Loan	1120	90.54	1014	1235468766.85	755272218.67	151	14.89	607720.92	
Total rows: 5 Query complete 00:00:00.382										

CRLF Ln 1120, Col 11

Total Recovery numeric	Recovery Rate (%) numeric
1430730.73	7.73
766424.27	8.81
2102866.53	13.12
91222.92	4.32
607720.92	7.99
CRLF Ln 1120, Col 11	